CS 7180: Claude Code Foundations

John Alexis Guerra Gomez
jguerra at northeastern.edu
Class: **johnguerra.co/classes/aiCoding_spring_2026**
Slides: **johnguerra.co/lectures/ai_assisted_coding**

What We'll Cover Today

1. Where We Are -- Week 10 checkpoint
2. How Claude Code Works
3. Installation & Setup
4. CLAUDE.md Deep Dive
5. Tool Use System & Permissions
6. Context Management
7. Thinking & Planning Modes
8. Hands-on Lab

Where We Are

> *Week 10 -- Entering Modality 3: Claude Code*

Recap: The Journey So Far

Modality 1 (Weeks 2-5): Claude Web -- architecture, prototyping, brainstorming
Modality 2 (Weeks 6-8): Antigravity -- production code in the IDE, Agile workflow, pair programming
Now -- Modality 3 (Weeks 10-12): Claude Code -- autonomous agents in your terminal

This Week: The Transition

- **P2 is due** -- your pair project wraps up
- **P3 team formation** -- form your 4-person teams this week
- **New tool, new mental model** -- Claude Code is fundamentally different from IDE chat

  P2 taught you pair workflow and IDE AI. P3 will push you into team-scale development with agentic tools.

How Claude Code Works

> *An agent, not a chatbot*

The Agentic Loop

Claude Code runs in a **continuous loop** -- it reads, thinks, acts, and verifies on its own.

Built-in Tools

Tool	Purpose	Example
Read	Read file contents	Read a config file to understand project setup
Edit	Make targeted edits	Replace a function, fix a bug
Write	Create or overwrite files	Generate a new component from scratch
Bash	Run shell commands	`npm test`, `git status`, `curl`
Glob	Find files by pattern	`**/*.test.ts` to find all test files
Grep	Search file contents	Find all imports of a module
WebFetch	Fetch URLs	Pull documentation, check an API

Claude Code selects and chains these tools autonomously to complete your task.

IDE Chat vs Claude Code

Dimension	IDE Chat (Antigravity)	Claude Code
Interface	Editor sidebar	Terminal
Interaction	Turn-based (you prompt, it responds)	Agentic loop (it keeps going)
File access	Current file + context you provide	Full project via tools
Shell access	Limited or none	Full Bash access
Workflow	You drive each step	It drives, you approve
Best for	Focused edits, inline changes	Multi-file tasks, refactoring, automation

Key Insight

Claude Code is an *agent*, not a chatbot.

- A chatbot waits for you after every response
- An agent takes initiative -- it reads files, runs tests, fixes errors, and keeps going until the task is done

You shift from **driving** to **supervising**.
This is a different skill. You need to get good at:

- Writing clear goals (not step-by-step instructions)
- Reviewing changes (not writing code yourself)
- Knowing when to intervene vs letting it continue

Installation & Setup

> *Three ways to start using Claude Code*

Installation Options

Method	How
Native Install (recommended)	`curl -fsSL https://claude.ai/install.sh | bash`
VS Code Extension	Search "Claude Code" in Extensions marketplace
Desktop App	Download from **claude.ai/download**

All three use the same engine. Native install auto-updates in the background.
Docs: **code.claude.com/docs**

Your First Session

```
cd ~/your-project
claude
```

What happens:

1. Claude Code scans your project structure
2. Reads any `CLAUDE.md` files it finds
3. Presents a prompt -- you type your task
4. It begins the agentic loop

First Time Prompts

Try something simple:

```
> Explain the architecture of this project
```

```
> What does the test suite cover?
```

Quick Demo Setup

For today's lab, make sure you have:

```
# Verify Node.js 18+
node --version

# Install Claude Code
npm install -g @anthropic-ai/claude-code

# Verify installation
claude --version

# Navigate to your project
cd ~/your-p3-repo
claude
```

CLAUDE.md Deep Dive

> *Persistent project instructions across sessions*

What Is CLAUDE.md?

A markdown file in your project root that Claude Code reads **every session**.
It is your project's **onboarding document for the AI** -- the same way you'd onboard a new teammate.

```
your-project/
  CLAUDE.md        <-- Claude Code reads this
automatically
  package.json
  src/
  tests/
```

Without a CLAUDE.md, Claude Code starts every session with zero project context. With one, it immediately knows your stack, conventions, and constraints.

What to Include

```
# CLAUDE.md

## Tech Stack
- Next.js 14, TypeScript 5.3, Tailwind CSS
- PostgreSQL with Prisma ORM
- Jest for unit tests, Playwright for E2E

## Build Commands
npm run dev        # Start development server
npm test           # Run all tests
npm run lint       # Lint check

## Architecture
- src/app/         -- Next.js App Router pages
- src/components/  -- Reusable React components
- src/lib/         -- Business logic, DB queries
- prisma/          -- Database schema and migrations

## Conventions
- All components use TypeScript strict mode
- API routes return { data, error } shape
- Tests live next to source files (*.test.ts)
- Never use `any` type -- use `unknown` and narrow

## Do NOT
- Modify the database schema without a migration
- Skip writing tests for new API routes
- Use client components unless interactivity requires
it
```

What NOT to Include

Too much content defeats the purpose. CLAUDE.md is read every session and consumes context.
Rule of thumb: Keep it concise. Context files that grow beyond a few hundred lines tend to see Claude deprioritize instructions — it has too much to track.

Avoid on CLAUDE.md

- **Entire codebases** -- Claude Code can read files on its own
- **Secrets or credentials** -- CLAUDE.md is committed to git
- **Long tutorials** -- Link to docs instead
- **Non-universal instructions** -- Claude wraps CLAUDE.md in "this may or may not be relevant", so non-universal rules get deprioritized

Rule of thumb: Keep it under 200 lines. Use @imports for detailed reference material.

@imports -- Referencing Other Files

Use `@import` to pull in content from other files without duplicating it:

```
# CLAUDE.md

## API Documentation
@docs/api-spec.md

## Database Schema
@prisma/schema.prisma

## Testing Patterns
@docs/testing-guide.md
```

This keeps your CLAUDE.md concise while giving Claude Code access to detailed reference material when it needs it.

The CLAUDE.md Hierarchy

Claude Code merges multiple CLAUDE.md files from different locations:

```
~/.claude/CLAUDE.md          (1) Global -- applies to
ALL projects
  |
  +-- your-project/
        CLAUDE.md            (2) Project root -- main
project context
        |
        +-- src/
        |     CLAUDE.md      (3) Directory -- src-
specific rules
        |
        +-- tests/
CLAUDE.md      (4) Directory -- test-specific rules
```

Merge order: Global -> Project root -> Subdirectories (most specific wins).
Use this to set global preferences (editor style, commit format) while keeping project-specific rules in each repo.

Auto-Memory

Claude Code can **remember things across sessions** on its own.
When you tell it something important, it may save it to CLAUDE.md automatically:

```
You: "Always use vitest instead of jest in this
project"
Claude Code: "I'll remember that." (adds to
CLAUDE.md)
```

You can also explicitly ask:

```
You: "Remember that our API rate limit is 100
req/min"
```

Check your CLAUDE.md periodically -- auto-memories accumulate and may need pruning.

The /init Command

Starting a new project? Let Claude Code generate the initial CLAUDE.md:

```
cd ~/your-new-project
claude

> /init
```

The /init Command (II)

Claude Code will:

1. Scan your project structure
2. Read package.json, config files, etc.
3. Generate a CLAUDE.md with tech stack, build commands, and architecture

Always review and edit the output. The auto-generated version is a starting point, not a finished product.

Tool Use System

> *How Claude Code selects tools and how you control it*

How Claude Selects Tools

Claude Code decides which tools to use based on the task:

- "Read the config file" -> **Read** tool
- "Find all TODO comments" -> **Grep** tool
- "Run the tests" -> **Bash** tool (`npm test`)
- "Fix the import in UserList.tsx" -> **Read** (understand) -> **Edit** (fix)
- "Add a new API route" -> **Read** (existing patterns) -> **Write** (new file) -> **Bash** (test it)

It chains tools naturally. A single prompt can trigger 10+ tool calls in sequence.

The Permission Model

Every tool call requires your approval by default:

```
Claude wants to run: npm test
Allow? (y/n/always)
```

Three permission levels:

Level	Behavior	When to Use
Ask every time	Approve each tool call	Learning, unfamiliar projects
Allowlist	Auto-approve specific tools/commands	Daily workflow, trusted commands
YOLO mode	Auto-approve everything	Sandboxed environments only

Configuring Allowlists

Add trusted commands to your settings so Claude Code doesn't ask every time:

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(npm test)",
      "Bash(git status)"
    ]
  }
}
```

Start restrictive, widen as trust builds.

Allow read-only tools first. Add write tools after you're comfortable reviewing changes.

Sandboxing

For maximum safety, run Claude Code inside a container:

```
# Docker sandbox
docker run -v $(pwd):/workspace -it claude-code-
sandbox

# Or use the built-in sandbox flag
claude --sandbox
```

Why sandbox?

- Claude Code has full Bash access -- it can `rm -rf`, install packages, make network calls
- A sandbox limits the blast radius of mistakes
- Required for YOLO mode in production environments

Best practice: Use sandboxing for unfamiliar codebases and during learning.

Context Management

> *The 200K token window is your most important resource*

Why Context Matters

Claude Code has a ~200K token context window. That sounds like a lot, but:

- A medium codebase can be 500K+ tokens
- Every file read, tool output, and conversation message consumes tokens
- When context fills up, Claude Code loses track of earlier information

Context is like RAM -- you need to manage it actively.

/clear -- Reset Context

```
> /clear
```

Wipes the conversation history. Claude Code starts fresh but still reads CLAUDE.md.

When to use:

- Switching to a completely different task
- Context is cluttered with irrelevant file contents
- Claude Code starts giving confused or inconsistent answers

Think of it as closing all browser tabs and starting over.

/compact -- Summarize and Compress

```
> /compact
```

Claude Code summarizes the conversation so far into a compressed form, freeing up context space.

When to use:

- Mid-task when context is getting full
- After a long exploration phase, before starting implementation
- You want to keep the thread but reclaim space

/compact keeps the important context. /clear throws everything away.

Checkpoints and /rewind

Claude Code creates **checkpoints** at key moments (before major edits, after commits).

```
> /rewind
```

Shows recent checkpoints and lets you roll back to a previous state -- both the conversation and file changes. **This is your undo button.** If Claude Code makes a bad edit or goes down the wrong path, `/rewind` takes you back.

Session Management

Resume your last session:

```
claude --continue
```

Picks up exactly where you left off -- same context, same files.

Browse and pick a session:

```
claude --resume
```

Shows a list of recent sessions. Useful when juggling multiple tasks.

Key insight: Sessions persist automatically. You don't lose work when you close the terminal.

Visualizing Your Context

Use `/context` to inspect what's in your session window:

```
> /context
```

Shows:

- Current token usage vs. limit
- What files and conversations are loaded
- Whether CLAUDE.md is active

Why it matters: Claude Code has a finite context window. When it fills up, older context gets dropped silently — you may get worse answers without knowing why. `/context` makes the invisible visible.

> *Rule of thumb: if responses start feeling "off," check `/context` first.*

Switching Models with /model

Switch mid-session:

```
> /model opus
> /model sonnet
> /model haiku
```

Or at launch:

```
claude --model claude-opus-4-6
```

Model	Best for
Opus	Complex refactors, architecture decisions
Sonnet	Daily coding, default balance
Haiku	Quick lookups, fast iteration

> ***Cost tip:*** *Use Sonnet by default. Switch to Opus when the task genuinely requires deeper reasoning.*

The "Context Stuffing" Anti-Pattern

Bad: Dumping everything into context upfront.

```
> Read all files in src/, then read all tests, then
read
> the README, then read package.json, then...
```

This fills your context window before you've done any real work.

Good: Let Claude Code pull in context as needed.

```
> Add input validation to the user registration
endpoint
```

Claude Code will read only the files it needs. Trust the agentic loop.

The Document-then-Implement Workflow

For complex tasks, use `/clear` as a **context reset tool**:

```
Phase 1: EXPLORE
> Explore the auth system and summarize findings

Phase 2: DOCUMENT
> Write your findings to docs/auth-analysis.md

Phase 3: /clear  (reset context -- findings are saved
in the file)

Phase 4: PLAN
> Read docs/auth-analysis.md and plan the refactoring

Phase 5: /clear  (reset context again)

Phase 6: IMPLEMENT
> Read docs/auth-analysis.md and implement the plan
```

Your **findings persist in files**, not in context. This lets you tackle tasks larger than the context window.

Best Practices for Context

1. **One task per session** -- start fresh for each distinct task
2. **Use /clear between tasks** -- don't carry stale context
3. **Use /compact for long sessions** -- preserve context, reclaim space
4. **Trust the agent** -- let it read files as needed, don't pre-load
5. **Keep CLAUDE.md concise** -- it's read every session, bloat wastes tokens
6. **Document then implement** -- save findings to files, /clear, then implement from the file

Thinking & Planning Modes

> *When to plan vs when to just do*

Normal Mode vs Plan Mode

Normal mode: Claude Code reads, edits, and executes immediately.

```
> Fix the broken import in UserList.tsx
```

Good for small, well-defined tasks.
Plan mode: Claude Code creates a plan first, then waits for your approval before executing.

```
> (plan) Refactor the authentication system to use
JWT
```

Good for large, multi-file changes where you want to review the approach first.

Switching Between Modes

Enter Plan mode:

- Type `(plan)` before your prompt
- Press `Shift+Tab` to toggle Plan mode

You can edit the plan before approving, redirect the approach, or cancel entirely.

In Plan mode, Claude Code will:

1. Analyze the codebase
2. Create a step-by-step plan
3. Show you the plan
4. Wait for your approval
5. Execute the plan

Extended Thinking

Claude Code uses **extended thinking** for complex reasoning -- it "thinks out loud" before responding. You'll see a thinking indicator while it works through the problem.

Helps with:

- Complex debugging (tracing through multiple files)
- Architecture decisions (weighing tradeoffs)
- Multi-step refactoring (ordering changes to avoid breakage)

You don't need to enable it -- Claude Code uses extended thinking automatically when the task warrants it.

When to Plan vs When to Do

Task	Mode	Why
Fix a typo	Normal	One file, obvious change
Add a test	Normal	Well-scoped, single file
New API endpoint	Plan	Multiple files (route, handler, test, types)
Refactor auth system	Plan	Cross-cutting, many dependencies
Debug a failing test	Normal	Start exploring, plan if complex
Set up CI/CD pipeline	Plan	Multiple config files, need to verify approach

Rule of thumb: If the task touches more than 3 files, use Plan mode.

Hands-on Lab

> *Get Claude Code running on your P3 repo*

Step 1: Install Claude Code

```
# Install
curl -fsSL https://claude.ai/install.sh | bash

# Verify
claude --version
```

If you already have it installed, update to the latest version:

```
claude update
```

Step 2: Initialize Your P3 Repo

```
cd ~/your-p3-repo
claude

> /init
```

Review the generated CLAUDE.md. Does it accurately capture your project?
Edit it to add:

- Your team's conventions
- Testing strategy
- Branch naming rules
- Anything from your P2 experience that you want to carry forward

Step 3: Write a Real CLAUDE.md

Based on what we covered, enhance the auto-generated CLAUDE.md:

- Add your tech stack with specific versions
- Add build and test commands
- Add architecture notes (where things live)
- Add conventions (naming, patterns, do's/don'ts)
- Add @imports for any reference docs

Commit it to your repo. This is a living document -- you'll update it throughout P3.

Step 4: The Explore-Plan-Implement-Commit Workflow

Try this four-step workflow on a simple task:

```
1. > Explore the codebase and explain the project
structure

2. > (plan) Add a health check endpoint at GET
/api/health
   >  that returns { status: "ok", timestamp: ... }

3. Review the plan, approve it, let Claude Code
implement

4. > Create a commit for the health check endpoint
```

This is the workflow you'll use daily in P3.

This Week's Deliverables

Due This Week

Project 2 -- due at the beginning of this week (should already be submitted)
Weekly Quiz 10 -- Claude Code Foundations concepts
P3 Team Formation:

- Form your 4-person teams
- Create your P3 GitHub repo
- Run `/init` and commit your initial CLAUDE.md

Next Week Preview

Week 11: Claude Code Workflows & TDD

- **Explore -> Plan -> Implement -> Commit** workflow in depth
- **TDD with Claude Code** -- red-green-refactor with AI assistance
- **Git & GitHub integration** -- commits, PRs, code review through Claude Code
- **CI/CD via Claude Code** -- GitHub Actions setup and automation
- **Non-interactive mode** -- scripting with `claude -p`, JSON output

HW4 (Claude Code Workflow & TDD) will be assigned.

Resources

Required Reading

Resource	URL
Claude Code in Action (Modules 1-3)	**anthropic.skilljar.com/claude-code-in-action**
Claude Code Overview	**code.claude.com/docs/en/overview**
CLAUDE.md Documentation	**code.claude.com/docs/en/claude-md**
Context Management Docs	**code.claude.com/docs/en/context-management**
Permissions & Security	**code.claude.com/docs/en/permissions**

Recommended Reading

Resource	URL
Claude Code Best Practices	**code.claude.com/docs/en/best-practices**
Writing a Good CLAUDE.md	**humanlayer.dev/blog/writing-a-good-claude-md**
Effective Context Engineering	**anthropic.com/engineering/effective-context-engineering-for-ai-agents**
My AI Coding Workflow (Addy Osmani)	**addyosmani.com/blog/ai-coding-workflow**
Trail of Bits: claude-code-config	**github.com/trailofbits/claude-code-config**

Speaker notes